

OSS_SURVEY

Open-Source Survey — ClickUp client libs + sync patterns

Field	Value
Revision	1
Last modified	2026-06-09T00:00:00Z
§11.4.74	Catalogue-first: prefer reuse/extend over reimplement.

Candidate ClickUp client libraries

Library	Lang	License	Maturity	Coverage	Verdict
raksul/go-clickup	Go	MIT	46★/25 forks, 113 commits, no tagged releases (pin a commit SHA)	Tasks CRUD, Lists, Folders, Spaces, Custom Fields set/remove, Tags add/remove, Webhooks create/get/update/delete, Members/Teams,	RE-COM-MEN-DED base

Library	Lang	License	Matur- ity	Coverage	Ver- dict
				Custom Task Types, Views, Com- ments, Check- lists, De- penden- cies, Get Filtered Team Tasks	
incident-io/ go-clickup	Go	(fork of rak- sul) MIT UNCONFIRMED:	fork	similar	fall- back if rak- sul stalls
Gui- tarbum722/ clickup-cli- ent-go	Go	UNCONFIRMED:	small	partial	not pre- ferred
theartofe- ducation/ clickup-go	Go	UNCONFIRMED:	small	partial	not pre- ferred
n8n ClickUp node / Zapi- er	(TS)	n8n fair- code	mature field- map- ping + web- hook UX pat- terns	full	mine for pat- terns, not as a lib

SOURCES: <https://github.com/raksul/go-clickup>, <https://pkg.go.dev/github.com/incident-io/go-clickup>, <https://github.com/Guitarbum722/clickup-client-go>, <https://github.com/theartofeducation/clickup-go>, <https://docs.n8n.io/integrations/builtin/credentials/clickup/>

Recommendation

Use `github.com/raksul/go-clickup/clickup` (MIT) as the **under-the-hood client**, pinned to a specific commit SHA (no releases = pin for reproducibility per §11.4.30/§11.4.77), and **extend it** for our gaps rather than reimplement (§11.4.74).

Rationale:

- **Go matches our stack** — `docs_chain` is Go (§11.4.106) and the workable-items single-source-of-truth binary is Go (§11.4.93). One language, one toolchain, shared SQLite layer.
- MIT license = clean to vendor/extend and push fixes upstream (§11.4.28 equal-codebase if we adopt it as an owned submodule, or replace directive during dev).
- Covers every endpoint we need (tasks, statuses-on-list, custom fields, tags, webhooks, filtered-team-tasks, members).

Gaps we MUST add on top (raksul does NOT provide)

1. **Rate-limit handling** — no 429 backoff in the lib. We wrap every call with `X-RateLimit-Reset`-driven backoff (RISK_ANALYSIS.md).
2. **Robust error typing** — thin error handling; we add typed errors + retry classification.
3. **Webhook signature verification** — lib creates webhooks but does not verify inbound `X-Signature`; we implement HMAC-SHA256 hex verify ourselves (API_REFERENCE.md §j).
4. **No CREATE-status / CREATE-custom-field-def** (no public endpoint anyway) — operator-seeds in UI.

Bidirectional-sync reference patterns (mined, not vendored)

SOURCES: n8n ClickUp trigger node + general sync literature (<https://docs.n8n.io/integrations/builtin/credentials/clickup/>); standard reconciliation practice.

- **Last-Write-Wins (LWW) by authoritative timestamp** — the simplest defensible mechanism for a 2-system sync; pick the side with the newer authoritative edit time. We adopt LWW keyed on ClickUp `date_updated` vs our SQLite `local_updated` (RISK_ANALYSIS.md).
- **Idempotency keys** — every outbound mutation carries a stable key so a retried POST/PUT after a network drop doesn't double-create.
- **Echo-suppression via a sync-marker** — stamp a per-item `last_synced_rev` into a ClickUp custom field (or a tag, or the loc-

al DB) so when our own write echoes back as a taskUpdated webhook we recognize "this is my edit" and don't loop. (The metadata-marker pattern.)

- **Webhook + reconciling full-sweep backstop** — webhooks are at-least-once/possibly-missed (API_REFERENCE.md §j); a periodic Get Filtered Team Tasks?date_updated_gt=<lastSeen> cron catches anything the webhook missed.
- **Dedupe by webhook history_items[].id** — ClickUp may re-deliver; dedupe on the history-item id, not just task_id.

Vector clocks are over-engineering for a 2-party (ClickUp ↔ our SQLite) sync — LWW with a well-defined newer-side rule + echo-suppression is sufficient and rock-solid. See RISK_ANALYSIS.md.